



南開大學
Nankai University

计算机学院
大数据计算及应用实验报告

电影评分预测系统

2310984 蔡子轩

2310675 陈宇昕

2026 年 6 月 29 日

目录

1	任务背景与问题定义	2
2	实验数据与评分机制	2
2.1	可用信息与数据来源	2
2.2	评分机制	2
3	方法设计	3
3.1	电影基础评分分布与信息回退策略	3
3.2	用户整体打分习惯修正	4
3.3	局部内容偏好修正	5
3.4	基于期望效用的整数评分选择	5
3.5	高置信短 Prompt 与边界样本 LLM 裁决	5
4	实验设计与优化过程	6
4.1	起点：直接让 LLM 自由评分	6
4.2	第一步改进：把评分判断移入代码	6
4.3	第二步改进：边界样本引入 LLM 二选一	7
5	实验结果与分析	7
6	局限性与展望	8

1 任务背景与问题定义

本实验的任务是根据用户的历史观影与评分记录，预测其对目标电影给出的 1-5 分整数评分。模型输出必须在最后一行包含 [Result:X]，格式错误将导致评测系统无法解析。

评测维度不止预测误差：除 MAE 和 RMSE 外，还包括精准命中率、误差不超过 1 分的比例、Token 效率和收益率，其中收益率占总分的 50%。这意味着一味堆长 Prompt 换取边际准确率提升并不合算，方案需要在预测准确性和 Token 开销之间找到平衡。后续将围绕这一目标，设计以统计预测为主、边界样本调用 LLM 为辅的混合方案。

2 实验数据与评分机制

2.1 可用信息与数据来源

实验中使用的信息分两类。第一类由平台在每次调用 generate_prompt(ctx) 时实时传入；第二类是根据训练集离线统计后压缩写入代码的静态表，以满足“只能提交单个函数、不能挂载外部文件”的限制。具体如表 1 所示。

表 1: 预测过程中使用的信息

类别	信息	来源	用途
实时传入	用户历史评分	ctx.user_history	计算用户均分和评分分布
	目标电影元数据	ctx.target_movie, ctx.movies_info	获取导演、类型、豆瓣评分等
	相似历史电影	ctx.get_similar_movies()	边界样本 LLM 判断时使用
离线统计	_H	训练集按电影 ID 分组，统计 1 到 5 分各档次数；电影 ID 和评分次数分别压缩为 RAW_PRIOR_KEYS 和 RAW_HIST_VALS，运行时解码还原	构造目标电影基础评分分布（最优先）
	_DBHIST	训练集按豆瓣评分档位分组，统计对应用户评分分布	无 _H 时以豆瓣分推断分布
	_GHIST、_GMEAN	训练集全局评分统计	信息不足时兜底；判断用户打分偏高或偏低
	_UT	根据平台评分规则构造的代理效用矩阵	从概率分布中选出期望效用最高的整数评分

注：RAW_PRIOR_KEYS 按顺序拼接电影 ID 后 8 位，RAW_HIST_VALS 按相同顺序以 36 进制字符保存各档评分次数，运行时通过 `int(c, 36)` 解码还原为 _H。

2.2 评分机制

评测同时考虑预测误差、命中情况、调用成本和推荐收益，具体包括 MAE、RMSE、精准命中率、误差不超过 1 分的比例、Token 效率和收益率，其中收益率占总分 50%。精准命中计 10 元收益，误差不超过 1 分计 2 元，误差超过 1 分仅计 0.1 元；每次调用还须扣除硬性成本和 Token 费用。因此方案需要在预测准确性和 Token 开销之间权衡，这也是后续“统计主预测、边界样本 LLM 裁决”设计的直接动因。

3 方法设计

本实验采用”统计预测为主、LLM 边界裁决为辅”的混合方案：先在代码中利用电影评分先验、用户历史和内容偏好构造概率分布，再根据平台评分规则选出最优整数评分；只有统计结果模糊的边界样本才调用 LLM，且输出空间被严格限制为两个相邻候选之一。整体流程如图 3.1 所示。

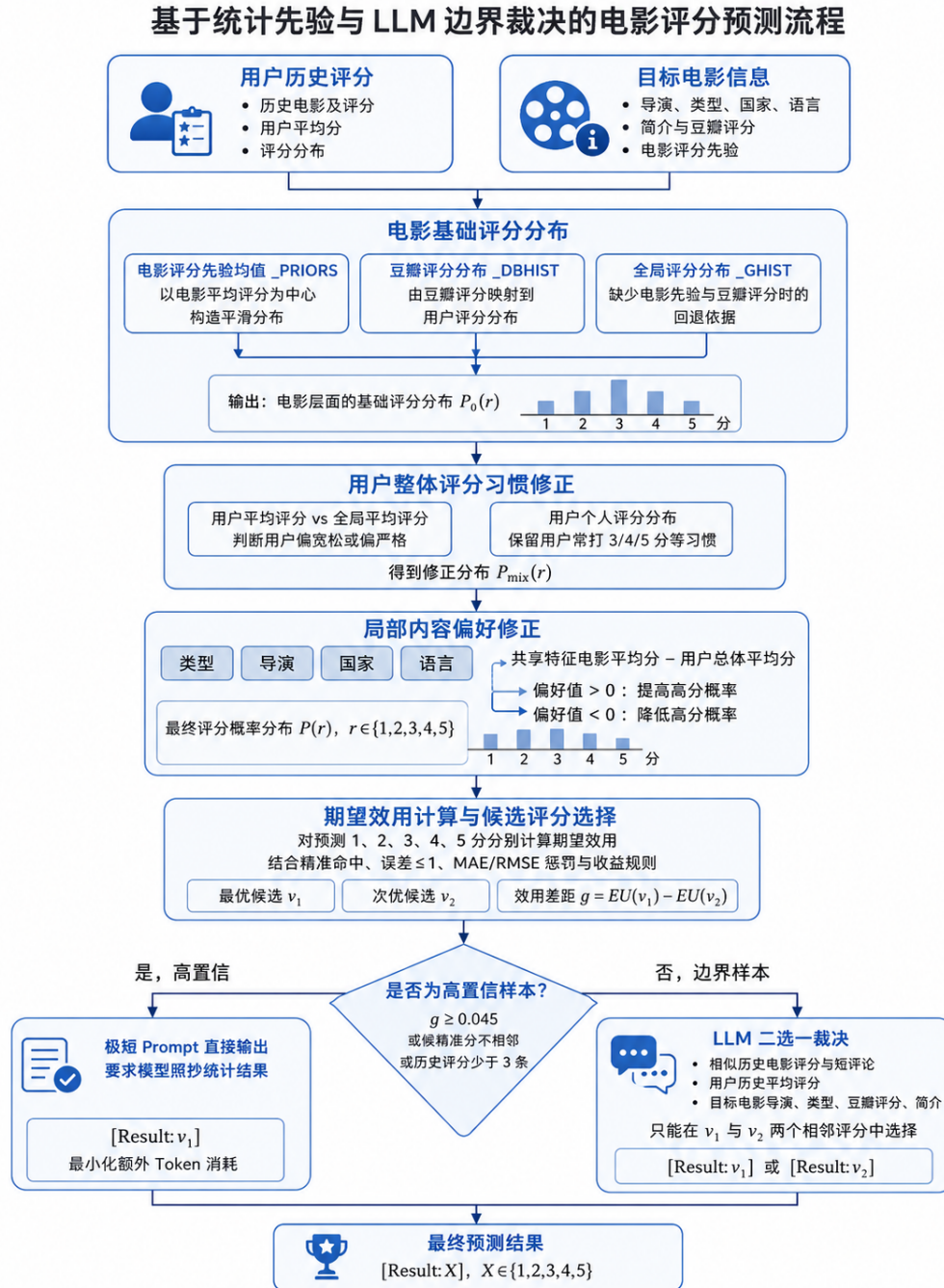


图 3.1: 电影评分预测整体流程

3.1 电影基础评分分布与信息回退策略

在不考虑具体用户偏好的情况下，系统首先需要判断目标电影大致处于哪个评分区间。代码将这一判断表示为离散概率分布 $P_0(r)$ ，其中 $r \in \{1, 2, 3, 4, 5\}$ 。

由于不同目标电影可获得的信息并不相同，基础评分分布按照信息可靠程度逐层构造。优先使用训练集中该电影已有的评分记录；若电影本身没有历史记录，再利用豆瓣评分提供整体口碑参考；对于两类信息都缺失的冷启动电影，则进一步参考当前用户对相似电影的评分；只有在上述信息均不足时，才退回训练集整体评分分布。

表 2: 不同信息完整度下的基础评分分布构造方式

可获得的信息	基础评分分布的构造方式
目标电影在 <code>_H</code> 中有历史评分记录	直接使用该电影在训练集中的 1-5 分评分直方图构造基础分布。若同时存在豆瓣评分，则再与豆瓣评分分布融合。
无 <code>_H</code> 记录，但存在豆瓣评分	将豆瓣评分映射到 5 分尺度，并结合 <code>_DBHIST</code> 中对应档位及相邻档位的用户评分分布，构造基础分布。
缺少上述两类信息，但用户历史中存在至少两部相似电影	调用 <code>ctx.get_similar_movies(8)</code> 获取标签接近的历史电影，以这些电影的用户评分均值为中心构造高斯分布。
相似电影也不足	退回训练集全局评分分布 <code>_GHIST</code> ，并以全局均分 <code>_GMEAN</code> 作为默认评分中心。

对于训练集中已有评分记录的电影，`_H` 保存了该电影被打 1-5 分的历史次数，由 `RAW_PRIOR_KEYS` 和 `RAW_HIST_VALS` 在运行时解码得到。若目标电影存在对应记录，则直接由该评分直方图构造电影基础分布 $P_H(r)$ 。若该电影同时带有豆瓣评分，则进一步构造豆瓣评分分布 $P_{db}(r)$ ，并按如下比例融合：

$$P_0(r) = 0.72 P_H(r) + 0.28 P_{db}(r).$$

当目标电影没有 `_H` 记录但存在豆瓣评分时，代码会先将 10 分制豆瓣评分除以 2，映射为 5 分尺度；随后查询 `_DBHIST` 中对应评分档位及相邻档位的用户评分分布。相邻档位采用递减权重参与聚合，避免电影评分刚好位于分档边界时，基础分布发生明显跳变。

相似电影在此处承担的是统计阶段的回退信息：代码只利用相似电影的评分均值构造目标电影的基础分布。后续若样本进入边界 LLM 分支，相似电影还会以“评分 + 短评论”的形式提供给模型，用于在两个相邻候选评分之间进行更细粒度的判断。

3.2 用户整体打分习惯修正

同一部电影，不同用户给出的分数可能系统性地偏高或偏低。代码用用户均分与全局均分之差衡量这种偏移：

$$\Delta_u = \bar{r}_u - \bar{r}_g,$$

其中 $\bar{r}_u$ 为当前用户历史均分， $\bar{r}_g = 3.651$ 为训练集全局均分。 $\Delta_u > 0$ 说明该用户整体偏宽松， $\Delta_u < 0$ 则偏严格。代码据此对 $P_0(r)$ 做指数加权调整，设电影基础均值为 μ_0 ：

$$P_s(r) \propto P_0(r) \cdot \exp(0.5 \cdot \text{SH} \cdot \Delta_u \cdot (r - \mu_0)).$$

除均分偏移外，代码还统计用户个人评分频数并平滑为分布 $P_u(r)$ ，保留“惯于打 4 分”或“喜欢给极端分”等个体习惯。最终将两者融合：

$$P_{\text{mix}}(r) = 0.35 P_u(r) + 0.65 P_s(r).$$

权重设计使电影本身的先验信息仍占主导，用户习惯只作适度修正。

3.3 局部内容偏好修正

整体均分偏移无法区分“用户对所有电影都给高分”和“用户特别偏爱某类电影”。代码进一步计算用户对目标电影各内容特征的偏好差值。设目标电影某特征值 q 对应的用户历史均分为 $\bar{r}_{u,q}$ ，则：

$$d_q = \bar{r}_{u,q} - \bar{r}_u.$$

$d_q > 0$ 表示用户对该特征有正向偏好， $d_q < 0$ 则相反。代码覆盖类型、导演、国家、语言四类特征，权重分别为 1.0、1.2、0.4、0.2（导演权重最高），加权求和得到总偏好调整量 A_u ，再对分布做最终修正：

$$P(r) \propto P_{\text{mix}}(r) \cdot \exp(0.15 A_u (r - \mu_0)).$$

归一化后得到用户对目标电影打 1 到 5 分的最终概率分布 $P(r)$ 。

3.4 基于期望效用的整数评分选择

得到 $P(r)$ 后，代码并不直接取概率最大的分数，而是计算每个候选评分 $v \in \{1, 2, 3, 4, 5\}$ 的期望效用：

$$\text{EU}(v) = \sum_{t=1}^5 P(t) \cdot U(v, t),$$

其中代理效用函数 $U(v, t)$ 根据误差 $e = |v - t|$ 定义：

$$U(v, t) = 0.0291 \cdot \text{Rev}(e) + B_0(e) + B_1(e) - 0.025e - 0.0263e^2,$$

$$\text{Rev}(e) = \begin{cases} 10, & e = 0, \\ 2, & e = 1, \\ 0.1, & e > 1, \end{cases} \quad B_0(e) = \begin{cases} 0.10, & e = 0, \\ 0, & \text{otherwise}, \end{cases} \quad B_1(e) = \begin{cases} 0.30, & e \leq 1, \\ 0, & e > 1. \end{cases}$$

$U(v, t)$ 不是对平台总分公式的完整复现，而是综合精准命中奖励、误差不超过 1 分奖励、MAE 和 RMSE 惩罚构造的近似决策函数。以概率集中在 3、4、5 分的样本为例：直接预测两端的 3 分或 5 分可能命中，但一旦偏离就会产生 2 分误差；预测 4 分虽不一定概率最高，却能将三种情况下的误差都控制在 1 分以内，期望效用反而更高。

计算完五个候选的期望效用后，取最优 v_1 和次优 v_2 ，记效用差距为：

$$g = \text{EU}(v_1) - \text{EU}(v_2).$$

3.5 高置信短 Prompt 与边界样本 LLM 裁决

满足以下任一条件时，视为高置信样本：

- $g \geq 0.045$ ，即最优候选明显优于次优；

- v_1 与 v_2 不相邻，不存在典型的“3 还是 4”问题；
- 用户历史评分少于 3 条，偏好信息不足以支撑 LLM 判断。

高置信样本使用极短 Prompt：系统提示词仅要求模型照抄，用户提示词直接给出 [Result: v_1]。这不是不调用模型，而是把额外输入压缩到最低，避免统计结果已经明确时模型自由发挥引入偏差。

不满足上述条件的样本为边界样本，此时 v_1 和 v_2 通常是相邻整数且效用接近。代码调用 `ctx.get_similar_movies(5)` 取标签最相近的历史电影，连同其评分和短评论一起传给 LLM；Prompt 中还包含用户历史均分、目标电影的导演、类型、豆瓣评分和截断简介。模型的输出被严格限制为只能在 v_1 和 v_2 之间二选一，例如只能输出 [Result:4] 或 [Result:5]，不能重新在 1~5 分间自由选择。

这一分工的实际效果是：绝大多数样本走短 Prompt 分支，Token 消耗极低；只有少数边界样本使用较长上下文，且 LLM 的判断空间已经被统计模块压缩到两个相邻选项，输出格式风险也随之降低。

4 实验设计与优化过程

4.1 起点：直接让 LLM 自由评分

最初的方案接近指导书提供的默认样例：把用户最近几条历史评分和目标电影的导演、类型、简介拼成 Prompt，让模型直接输出 1~5 分。这种做法实现简单，也确实能利用 LLM 对电影内容的语义理解能力。

但跑了几组测试后发现两个问题。第一，Token 消耗不低——每条样本都要传完整的用户历史和电影简介，而收益率对调用成本很敏感，长 Prompt 带来的准确率提升有限，整体收益率并不好看。第二，输出不够稳定：对于一些统计信号很明确的样本，例如目标电影在训练集中几乎清一色被打 4 分、当前用户也偏向高分，LLM 有时仍会因为简介里某句负面描述而给出偏差较大的评分，反而比直接用统计结果更差。

4.2 第一步改进：把评分判断移入代码

意识到上述问题后，调整的核心思路是：LLM 不适合对每个样本都从头推理，结构化的评分信息用统计方法处理更稳定。

于是把评分判断的主体逻辑移入 Python 代码，分层构造用户对目标电影打 1~5 分的概率分布。具体流程为：首先查询训练集中该电影的历史评分直方图 `_H`，若存在则直接以其构造基础分布，并与豆瓣分布按 6:4 融合；若 `_H` 中无记录但有豆瓣评分，则查询 `_DBHIST` 中对应档位的用户评分分布；若两者均无，则看当前用户对相似电影的历史评分；相似电影也不足时，退回全局分布 `_GHIST` 兜底。在此基础上，再叠加当前用户整体打分偏移（用户均分与全局均分之差）以及类型、导演、国家、语言等局部偏好修正，得到最终概率分布 $P(r)$ 。

得到概率分布后，直接取概率最大的分数并不是最优策略，还需要考虑平台评测的给分逻辑。平台对精准命中、误差不超过 1 分和较大误差的处理差异显著：精准命中收益 10 元，误差 1 分收益 2 元，误差超过 1 分仅 0.1 元。这意味着当概率分散在相邻几个分数时，选择居中的整数往往比选择概率略高的端点更划算。因此引入期望效用函数 $U(v, t)$ ，对每个候选评分计算其在当前概率分布下的期望收益，最终选出期望效用最高的整数 v_1 ，而非概率最高的整数。

这一版本的 LLM 角色退化为格式化输出——Prompt 极短，只要求模型照抄 v_1 。Token 消耗大幅下降，Token 效率得分随之明显提升；同时因为大多数样本的预测结果不再依赖模型自由发挥，输出

稳定性也有所改善。统计模块内部的几个关键参数——融合权重、用户习惯修正强度、内容偏好调整系数——均在测试集上经过多轮对比后固定下来。

4.3 第二步改进：边界样本引入 LLM 二选一

纯统计方案在大多数样本上表现稳定，但有一类情况处理得不好：最优候选 v_1 和次优候选 v_2 是相邻整数，且期望效用差距极小。例如 EU(4) 和 EU(5) 相差不到 0.01，说明统计信息本身无法可靠区分这两个评分，强行选一个反而可能引入系统性偏差。

这类边界样本恰好是 LLM 擅长处理的场景：给它目标电影的简介、导演和类型，以及用户对相似电影的评分和短评论，模型能够利用语义信息做出比统计更细粒度的判断。于是在统计分支之外引入边界样本分支：当两个相邻候选的效用差距低于阈值 0.045、且用户历史不少于 3 条时，调用 `ctx.get_similar_movies(5)` 取标签最接近的历史电影，连同相关文本一起传给 LLM，但输出空间被严格限制为只能在 v_1 和 v_2 之间二选一，不能重新在 1-5 分间自由选择。

这一改动对 Token 总量影响有限，因为触发边界分支的样本只占少数；但对精准命中率有实际帮助，相邻评分的细粒度区分是整体得分的主要瓶颈之一。最终方案的分工是：统计模块覆盖绝大多数样本，LLM 只介入统计已经把候选缩小到两个、但仍难以区分的少数情况，两者各司其职。

5 实验结果与分析

最终方案在评分集上的综合得分为 98.7，各项指标见表 3、表 4 和表 5。

表 3: 评分集准确性指标

指标	数值
MAE	0.6869
RMSE	0.9535
精准命中率	41.41%
误差不超过 1 分的比例	90.91%

表 4: 评分集成本与收益指标

指标	数值
输入 Token 总量	9,787
输出 Token 总量	600
总成本	106.09 元
总收益	508.90 元
收益率	379.7%
最终评分集得分	98.7

表 5: 评分集各项指标得分

指标	得分
MAE 得分	82.8
RMSE 得分	76.2
精准命中得分	41.4
误差不超过 1 分得分	90.9
Token 效率得分	100.0
收益率得分	82.8

Token 效率得分满分 (100.0)，总输入 Token 仅 9,787，总输出 Token 为 600，印证了高置信短 Prompt 策略的有效性——绝大多数样本走极短 Prompt 分支，只有边界样本才使用较长上下文，整体调用成本控制在预期范围内。收益率 379.7% 也说明方案没有用堆砌 Token 换取边际准确率，成本与收益的比例合理。

误差不超过 1 分的比例达到 90.91%，得益于期望效用选分机制在概率分散时倾向于选择风险更低的居中整数，有效压制了大误差样本。相比之下，精准命中率 41.41% 仍是当前方案的主要瓶颈：3/4 分和 4/5 分之间的边界样本，统计信号和 LLM 语义判断都难以稳定区分，是后续优化的重点方向。

6 局限性与展望

最终得分 98.7，距离满分仍有小幅差距，主要瓶颈集中在相邻评分的精准区分上。以下几点是当前方案比较明显的局限，也是后续可以改进的方向，但受限于时间和测试次数，未能在本次实验中进一步验证。

第一，`_H` 保存的是完整评分直方图，但参数调整过程中部分地方仍依赖经验设定——融合权重、偏好调整系数、高置信阈值 0.045 等均未经过系统的参数搜索，只是在测试集上多轮对比后人工固定。不同参数组合对最终得分的影响尚不明确。

第二，高置信分支会把统计阶段的判断直接锁定，LLM 没有机会用简介或评论信息纠偏；边界分支虽然引入了语义判断，但输出被限制在 v_1 和 v_2 之间，若两个候选本身都偏离真实评分，模型也无法跳出这个范围。稳定性和灵活性之间的张力在当前方案里没有完全解决。

第三，对于既无 `_H` 记录、又无豆瓣评分的冷启动电影，只能退回全局分布兜底，个性化程度很低。相似历史电影目前只用于边界样本的 LLM 判断，未被纳入统计预测阶段的回退链，是一个可以利用但尚未充分利用的信息来源。